МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ДЕРЖАВНИЙ УНІВЕРСИТЕТ «ЖИТОМИРСЬКА ПОЛІТЕХНІКА»
Факультет інформаційно-комп’ютерних технологій
Кафедра інженерії програмного забезпечення

ЗВІТ

з лабораторної роботи № 3
з дисципліни «Розробка мобільних додатків»
на тему:
«Expo Router: кастомізація навігаційних компонентів, нативні модальні вікна та віртуалізація списків»

Виконала: студентка групи ІПЗ-24-2
Дзядзін Марина Василівна
Перевірив:
Нерода Сергій Іванович

Житомир - 2026

Мета: закріплення практичних навичок використання файлової маршрутизації Expo Router, проєктування ієрархічних систем навігації та застосування компонента `VirtualizedList` для оптимізованого рендерингу списків даних.

ЗАВДАННЯ

Створити прототип модуля адміністрування порталу новин на основі файлової маршрутизації Expo Router з кастомізованим боковим меню, нативними модальними вікнами для створення та фільтрації вмісту і віртуалізованими списками. Вимоги:

- **Авторизація** - екран входу в систему.
- **Панель адміністрування** - група маршрутів для службових розділів.
- **Збереження даних** - локальні масиви об'єктів новин, користувачів і системних логів (щонайменше по 50 записів).
- **Custom Drawer** - навігатор `<Drawer>` з власним компонентом `drawerContent`: профіль адміністратора (аватар, ім'я, пошта), пункти «Дашборд», «Новини», «Користувачі», «Системні логи» та кнопка «Вийти» внизу.
- **Дашборд** - кількість новин, користувачів і логів з актуалізацією через `useFocusEffect`.
- **Новини** - `VirtualizedList` з параметрами оптимізації; поля: заголовок, категорія, статус, URL зображення, текст, дата створення; екран деталей `/news/[id]` з `useLocalSearchParams`.
- **Створення, редагування, видалення** - екран із `presentation: 'modal'`, видалення з підтвердженням через `Alert.alert()`.
- **Користувачі** - `VirtualizedList` з ролями та статусами (Активний / Заблокований) і можливістю зміни статусу.
- **Журнал логів** - `VirtualizedList`; запис додається лише при вході, виході, створенні, редагуванні, видаленні новини та зміні статусу користувача.

– **Фільтрація** - панелі фільтрації новин і користувачів у режимі `presentation: 'formSheet'` з `sheetAllowedDetents`.
– Розмістити код у репозиторії «MobileLabsRN2026» у папці lab3 з README.md та відповідями на контрольні запитання.

ХІД РОБОТИ

1. Створення проєкту

Проєкт створено на шаблоні Expo SDK 57, демонстраційний код прибрано. Для запобігання перекриттю інтерфейсу вирізами екрана, статус-баром і системною панеллю використано бібліотеку `react-native-safe-area-context` (`SafeAreaProvider`, `SafeAreaView`, `useSafeAreaInsets`).

Лістинг 3.1 - Створення проєкту

```
npx create-expo-app@latest lab3 --template default@sdk-57
cd lab3
npm run reset-project
npx expo install @expo/vector-icons @expo-google-fonts/montserrat
```

2. Архітектура маршрутів

Кореневий Stack містить екран входу, групу `(admin)` з навігатором Drawer та три модальні екрани. Розділ новин має власний вкладений Stack для переходу від списку до деталей за адресою `/news/[id]`.

Лістинг 3.2 - Структура проєкту lab3

```
lab3/
├── app/
│   ├── _layout.jsx            # провайдери, кореневий Stack, Stack.Protected,
modal/formSheet
│   ├── login.jsx              # екран входу
│   ├── news-form.jsx          # presentation: 'modal' - створення / редагування
новини
│   ├── news-filter.jsx        # presentation: 'formSheet' - фільтр новин
│   ├── users-filter.jsx       # presentation: 'formSheet' - фільтр користувачів
│   └── (admin)/
│       ├── _layout.jsx        # Drawer + drawerContent + screenOptions.header
│       ├── index.jsx          # дашборд (useFocusEffect)
│       ├── users.jsx          # VirtualizedList користувачів
│       ├── logs.jsx           # VirtualizedList журналу
│       └── news/
│           ├── _layout.jsx    # вкладений Stack з власним хедером
│           ├── index.jsx      # VirtualizedList новин
│           └── [id].jsx       # деталі новини /news/[id]
├── components/
│   ├── AdminDrawerContent.jsx # власний вміст бокового меню
```

```
│   ├── AdminHeader.jsx        # власний хедер
│   ├── FilterSheet.jsx        # спільний вміст панелей фільтрації
│   ├── ui.jsx                 # Badge, ChipSelect, кнопки, Avatar, confirmAction
│   └── BrandMark.jsx
├── context/AdminContext.jsx   # дані, CRUD, журнал, фільтри, авторизація
└── data/seed.js               # генерація масивів
```

3. Локальні дані та глобальний стан

Початкові дані генеруються в модулі `data/seed.js`: 60 новин, 60 користувачів і 50 записів журналу. Новина містить поля `title`, `category`, `status` (Опубліковано, Чернетка, На перевірці), `imageUrl`, `text` і `createdAt`.

Лістинг 3.3 - data/seed.js (фрагмент: генерація новин)

```
export function createSeedNews(count = 60) {
  return Array.from({ length: count }, (_, i) => {
    const id = String(count - i);
    return {
      id,
      title: TITLES[i % TITLES.length],
      category: NEWS_CATEGORIES[i % NEWS_CATEGORIES.length],
      status: NEWS_STATUSES[(i * 7) % 3 === 0 ? 0 : i % 3],
      imageUrl: `https://picsum.photos/seed/admin-news-${id}/800/500`,
      text: BODY,
      createdAt: new Date(NOW - i * 9 * HOUR).toISOString(),
    };
  });
}
```

Усі масиви зберігаються в контексті `AdminContext` (`useState`). Контекст надає функції авторизації, CRUD-операції з новинами, зміну статусу користувача та стан фільтрів. Функція `addLog` додає запис у журнал і викликається лише з шести операцій, визначених завданням.

Лістинг 3.4 - context/AdminContext.jsx (фрагмент)

```
export function AdminProvider({ children }) {
  const [admin, setAdmin] = useState(null);
  // Локальні масиви об'єктів - "база даних" прототипу
  const [news, setNews] = useState(createSeedNews);
  const [users, setUsers] = useState(createSeedUsers);
  const [logs, setLogs] = useState(createSeedLogs);
  const [newsFilter, setNewsFilter] = useState(DEFAULT_NEWS_FILTER);
  const [userFilter, setUserFilter] = useState(DEFAULT_USER_FILTER);
  const seq = useRef(1000);


  const nextId = () => String(++seq.current);


  // Новий запис у журнал додається лише для шести системних подій (LOG_TYPES)
  const addLog = useCallback((type, details, actor) => {
    if (!LOG_TYPES[type]) return;
    setLogs((prev) => [
```

```
      { id: `log-${Date.now()}-${Math.random().toString(36).slice(2, 7)}`, type,
details, actor, createdAt: new Date().toISOString() },
      ...prev,
    ]);
  }, []);


  const login = useCallback(
    (email, password) => {
      const cleanEmail = email.trim().toLowerCase();
      if (cleanEmail !== ADMIN_CREDENTIALS.email || password !==
ADMIN_CREDENTIALS.password) {
        return { ok: false, error: 'Невірна пошта або пароль' };
      }
      setAdmin({ name: student.fullName, email: cleanEmail, role: 'Адміністратор' });
      addLog('LOGIN', cleanEmail, cleanEmail);
      return { ok: true };
    },
    [addLog]
  );


  const logout = useCallback(() => {
    if (admin) addLog('LOGOUT', admin.email, admin.email);
    setAdmin(null);
  }, [admin, addLog]);


  const createNews = useCallback(
    (data) => {
      const item = { ...data, id: nextId(), createdAt: new Date().toISOString() };
      setNews((prev) => [item, ...prev]);
      addLog('NEWS_CREATE', `«${item.title}»`, admin?.email);
      return item;
    },
    [admin, addLog]
  );


  const updateNews = useCallback(
    (id, data) => {
      setNews((prev) => prev.map((n) => (n.id === id ? { ...n, ...data } : n)));
      addLog('NEWS_UPDATE', `«${data.title}»`, admin?.email);
    },
    [admin, addLog]
  );


  const deleteNews = useCallback(
    (id) => {
      const item = news.find((n) => n.id === id);
      setNews((prev) => prev.filter((n) => n.id !== id));
      if (item) addLog('NEWS_DELETE', `«${item.title}»`, admin?.email);
    },
    [news, admin, addLog]
  );


  const toggleUserStatus = useCallback(
    (id) => {
      const user = users.find((u) => u.id === id);
      if (!user) return;
      const status = user.status === 'Активний' ? 'Заблокований' : 'Активний';
      setUsers((prev) => prev.map((u) => (u.id === id ? { ...u, status } : u)));
      addLog('USER_STATUS', `${user.name} → ${status}`, admin?.email);
    },
    [users, admin, addLog]
  );
```

4. Авторизація та захист маршрутів

У кореневому макеті використано `Stack.Protected`: екран входу доступний лише неавторизованому користувачу, а панель адміністрування та модальні вікна - лише авторизованому. Після входу чи виходу навігатор автоматично перемикає доступні екрани. У цьому ж файлі задано режими показу модальних вікон: `modal` для форми новини та `formSheet` з `sheetAllowedDetents` для панелей фільтрації.

Лістинг 3.5 - app/_layout.jsx (фрагмент)

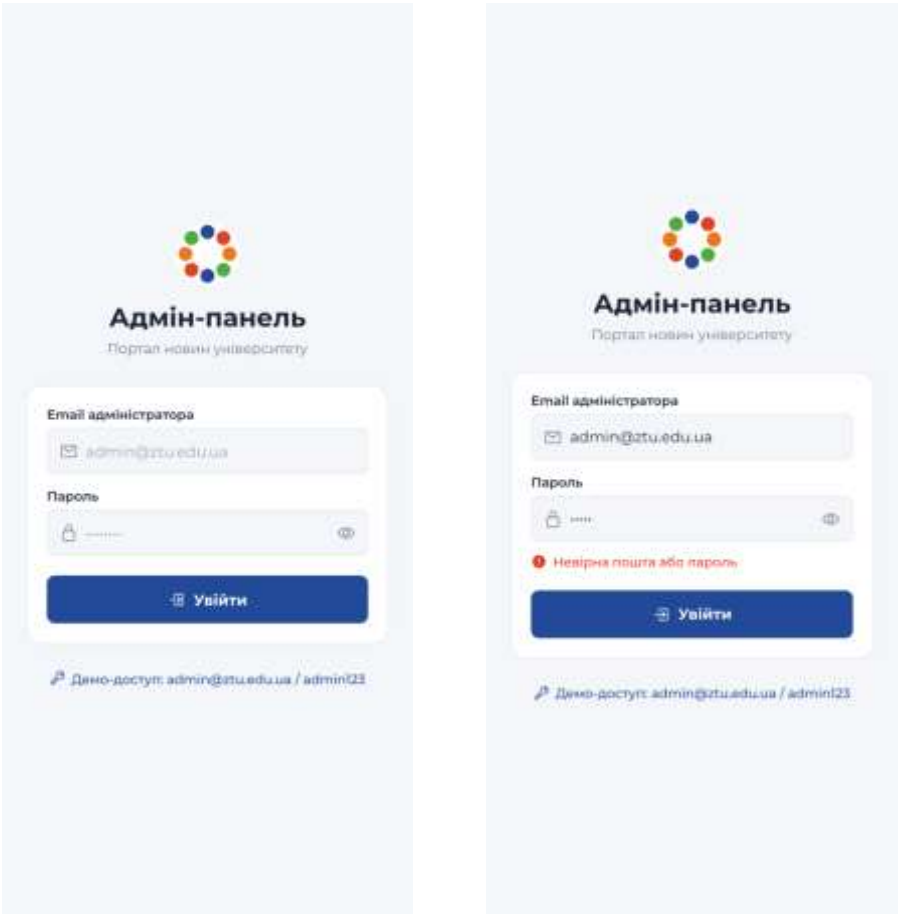
```
const sheetOptions = {
  presentation: 'formSheet',
  sheetAllowedDetents: [0.6, 0.95], // висота панелі: 60% або майже весь екран
  sheetInitialDetentIndex: 0,
  sheetGrabberVisible: true,
  sheetCornerRadius: 24,
  contentStyle: { backgroundColor: colors.surface },
};

function RootNavigator() {
  const { isAuthenticated } = useAdmin();

  return (
    <Stack screenOptions={{ headerShown: false }}>
      {/* Публічний екран: доступний лише неавторизованим */}
      <Stack.Protected guard={!isAuthenticated}>
        <Stack.Screen name="login" />
      </Stack.Protected>

      {/* Захищена частина: панель адміністрування + модальні вікна */}
      <Stack.Protected guard={isAuthenticated}>
        <Stack.Screen name="(admin)" />
        <Stack.Screen name="news-form" options={{ presentation: 'modal' }} />
        <Stack.Screen name="news-filter" options={sheetOptions} />
        <Stack.Screen name="users-filter" options={sheetOptions} />
      </Stack.Protected>
    </Stack>
  );
}
```

Екран входу перевіряє облікові дані адміністратора (демо-доступ: `admin@ztu.edu.ua` / `admin123`) і виводить повідомлення про помилку при невірних даних.



а) екран входу б) невірні облікові дані

Рисунок 3.1 - Авторизація

5. Кастомізоване бокове меню (drawerContent)

Навігатору `<Drawer>` через проп `drawerContent={(props) => <AdminDrawerContent {...props} />}` передано власний компонент. Він містить блок профілю адміністратора (аватар з ініціалами, ім'я, пошта, роль), стандартні пункти навігації (`DrawerItemList` усередині `DrawerContentScrollView`) та закріплену внизу кнопку «Вийти». Через `screenOptions.header` для всіх екранів підключено власний хедер.

Лістинг 3.6 - app/(admin)/_layout.jsx

```
import { Ionicons } from '@expo/vector-icons';
import { Drawer } from 'expo-router/drawer';
import AdminDrawerContent from '../../components/AdminDrawerContent';
import AdminHeader from '../../components/AdminHeader';
import { colors, fonts } from '../../constants/theme';


// Група маршрутів (admin): службові розділи керування порталом новин
export default function AdminLayout() {
  return (
    <Drawer
```

```
      drawerContent={(props) => <AdminDrawerContent {...props} />}
      screenOptions={{
        header: (props) => <AdminHeader {...props} />,
        drawerActiveTintColor: colors.blue,
        drawerActiveBackgroundColor: '#E8EEF9',
        drawerInactiveTintColor: colors.text,
        drawerLabelStyle: { fontFamily: fonts.semibold, fontSize: 15 },
        drawerItemStyle: { borderRadius: 10 },
        sceneStyle: { backgroundColor: colors.background },
      }}
    >
      <Drawer.Screen
        name="index"
        options={{
          title: 'Дашборд',
          drawerIcon: ({ color, size }) => <Ionicons name="grid-outline" size={size}
color={color} />,
        }}
      />
      <Drawer.Screen
        name="news"
        options={{
          title: 'Новини',
          headerShown: false, // у розділу новин власний Stack із хедером
          drawerIcon: ({ color, size }) => <Ionicons name="newspaper-outline"
size={size} color={color} />,
        }}
      />
      <Drawer.Screen
        name="users"
        options={{
          title: 'Користувачі',
          drawerIcon: ({ color, size }) => <Ionicons name="people-outline" size={size}
color={color} />,
        }}
      />
      <Drawer.Screen
        name="logs"
        options={{
          title: 'Системні логи',
          drawerIcon: ({ color, size }) => <Ionicons name="document-text-outline"
size={size} color={color} />,
        }}
      />
    </Drawer>
  );
}
```

Лістинг 3.7 - components/AdminDrawerContent.jsx (без стилів)

```
import { Ionicons } from '@expo/vector-icons';
import { DrawerContentScrollView, DrawerItemList } from 'expo-router/drawer';
import { Pressable, StyleSheet, Text, View } from 'react-native';
import { useSafeAreaInsets } from 'react-native-safe-area-context';
import { colors, fonts, student } from '../constants/theme';
import { useAdmin } from '../context/AdminContext';
import BrandMark from './BrandMark';
import { Avatar } from './ui';


// Власний вміст бокового меню: профіль адміністратора, пункти навігації, кнопка
"Вийти"
export default function AdminDrawerContent(props) {
  const { admin, logout } = useAdmin();
  const insets = useSafeAreaInsets();


  return (
```

```
    <View style={{ flex: 1 }}>
      <View style={[styles.profile, { paddingTop: insets.top + 20 }]}>
        <View style={styles.brandRow}>
          <BrandMark size={28} />
          <Text style={styles.brandText}>Адмін-панель новин</Text>
        </View>
        <Avatar name={admin?.name ?? 'A'} size={64} color={colors.green} />
        <Text style={styles.name}>{admin?.name}</Text>
        <Text style={styles.email}>{admin?.email}</Text>
        <View style={styles.roleBadge}>
          <Ionicons name="shield-checkmark" size={12} color="#fff" />
          <Text style={styles.roleText}>{admin?.role}</Text>
        </View>
      </View>

      <DrawerContentScrollView {...props} contentContainerStyle={{ paddingTop: 12 }}>
        <DrawerItemList {...props} />
      </DrawerContentScrollView>

      <View style={[styles.bottom, { paddingBottom: insets.bottom + 12 }]}>
        <Pressable
          onPress={logout}
          style={({ pressed }) => [styles.logout, pressed && { opacity: 0.7 }]}
          accessibilityLabel="Вийти"
        >
          <Ionicons name="log-out-outline" size={22} color={colors.red} />
          <Text style={styles.logoutText}>Вийти</Text>
        </Pressable>
        <Text style={styles.footer}>
          ЛР №3 · {student.fullName}, {student.group}
        </Text>
      </View>
    </View>
  );
}

// ... оголошення стилів (StyleSheet.create) опущено
```

6. Власний хедер (screenOptions.header)

Компонент `AdminHeader` отримує від навігатора `navigation`, `route`, `options` і `back`. Якщо в стеку є попередній екран, показується кнопка «Назад», інакше - кнопка відкриття меню (`DrawerActions.toggleDrawer()`). Хедер виводить заголовок, підзаголовок-лічильник (`headerSubtitle`) та дії праворуч (`headerRight`), які кожен екран задає самостійно.

Лістинг 3.8 - components/AdminHeader.jsx (без стилів)

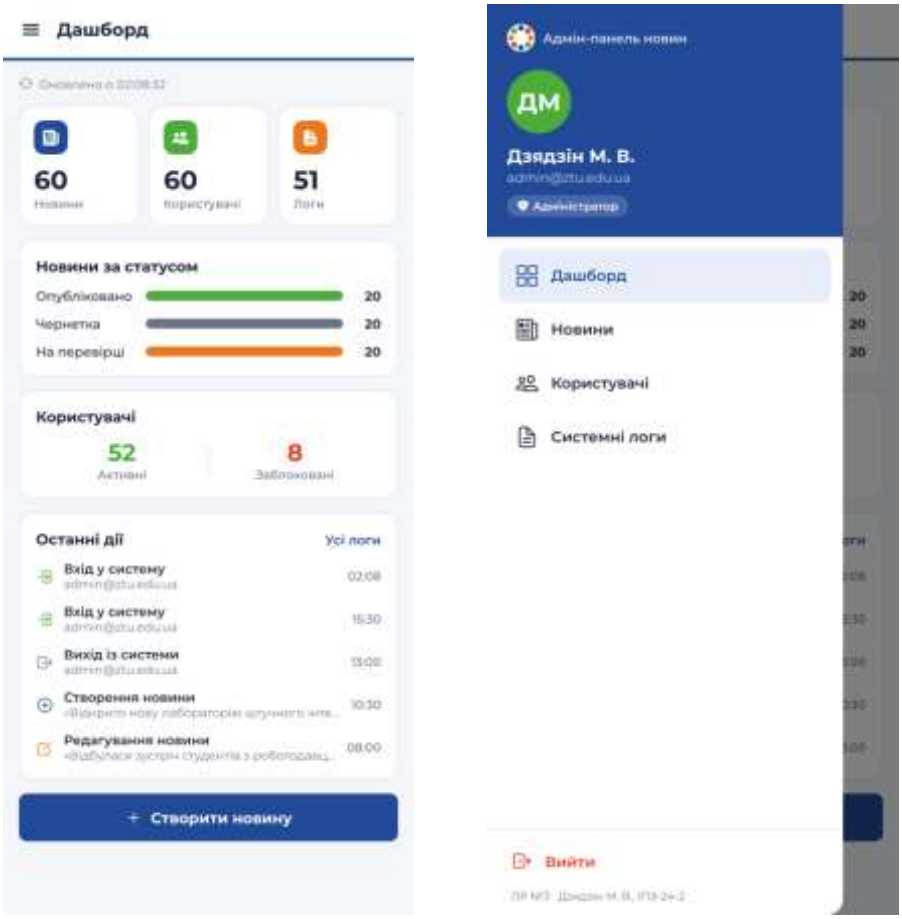
```
import { DrawerActions } from 'expo-router/react-navigation';
import { StyleSheet, Text, View } from 'react-native';
import { useSafeAreaInsets } from 'react-native-safe-area-context';
import { colors, fonts } from '../constants/theme';
import { HeaderIconButton } from './ui';

// Власний хедер, що передається через screenOptions.header
// (і для Drawer, і для вкладеного Stack). Якщо є попередній екран - кнопка "Назад",
// інакше - кнопка відкриття бокового меню.
export default function AdminHeader({ navigation, options, route, back }) {
```

```
  const insets = useSafeAreaInsets();
  const title = options.title ?? route.name;


  return (
    <View style={[styles.header, { paddingTop: insets.top + 6 }]}>
      {back ? (
        <HeaderIconButton icon="arrow-back" label="Назад" onPress={navigation.goBack}
/>
      ) : (
        <HeaderIconButton icon="menu" label="Меню" onPress={() =>
navigation.dispatch(DrawerActions.toggleDrawer())} />
      )}
      <View style={{ flex: 1 }}>
        <Text style={styles.title} numberOfLines={1}>
          {title}
        </Text>
        {options.headerSubtitle ? <Text
style={styles.subtitle}>{options.headerSubtitle}</Text> : null}
      </View>
      <View style={styles.right}>{options.headerRight?.({ tintColor: colors.text
})}</View>
    </View>
  );
}

// ... оголошення стилів (StyleSheet.create) опущено
```



а) дашборд б) бокове меню

Рисунок 3.2 - Дашборд і кастомізоване бокове меню

7. Дашборд і useFocusEffect

Дашборд відображає загальну кількість новин, користувачів і логів, розподіл новин за статусами, кількість активних і заблокованих користувачів та останні дії. Статистика перераховується в хуку `useFocusEffect` щоразу, коли екран отримує фокус (повернення з іншого розділу чи закриття модального вікна); час останнього оновлення показується на екрані.

Лістинг 3.9 - app/(admin)/index.jsx (фрагмент)

```
function computeStats(news, users, logs) {
  return {
    newsTotal: news.length,
    usersTotal: users.length,
    logsTotal: logs.length,
    byStatus: NEWS_STATUSES.map((s) => ({ status: s, count: news.filter((n) =>
n.status === s).length })),
    activeUsers: users.filter((u) => u.status === 'Активний').length,
    blockedUsers: users.filter((u) => u.status === 'Заблокований').length,
    recentLogs: logs.slice(0, 5),
    updatedAt: new Date(),
  };
}


export default function DashboardScreen() {
  const router = useRouter();
  const { news, users, logs } = useAdmin();
  const [stats, setStats] = useState(null);


  // useFocusEffect: статистика перераховується щоразу, коли екран отримує фокус
  // (повернення з інших розділів, закриття модального вікна тощо)
  useFocusEffect(
    useCallback(() => {
      setStats(computeStats(news, users, logs));
    }, [news, users, logs])
  );


  if (!stats) return null;
```

8. Список новин на VirtualizedList

Список новин реалізовано безпосередньо на `VirtualizedList`, якому передаються функції `getItemCount` і `getItem`, а також `getItemLayout` (фіксована висота рядка), `keyExtractor`, `initialNumToRender`, `maxToRenderPerBatch` і `windowSize`. Рядок мемоїзовано через `memo`. У хедері розміщено кнопки фільтра та створення новини, а над списком - панель активних фільтрів із кнопкою «Скинути».

Лістинг 3.10 - app/(admin)/news/index.jsx (без стилів)

```
import { Ionicons } from '@expo/vector-icons';
import { Stack, useRouter } from 'expo-router';
import { memo, useCallback, useMemo } from 'react';
import { Image, Pressable, StyleSheet, Text, View, VirtualizedList } from 'react-
native';
import { Badge, HeaderIconButton } from '../../../components/ui';
import { colors, fonts, radius } from '../../../constants/theme';
import { DEFAULT_NEWS_FILTER, filterNews, useAdmin } from
'../../../context/AdminContext';
import { formatDate } from '../../../data/seed';


const ITEM_HEIGHT = 108; // висота картки + відступ - для getItemLayout
const CARD_HEIGHT = 96;
const PADDING = 16;


const NewsRow = memo(function NewsRow({ item, onPress }) {
  return (
    <Pressable onPress={() => onPress(item.id)} style={({ pressed }) => [styles.card,
pressed && { opacity: 0.85 }]}>
      <Image source={{ uri: item.imageUrl }} style={styles.image} />
      <View style={styles.body}>
        <View style={styles.topRow}>
          <Text style={styles.category}>{item.category.toUpperCase()}</Text>
          <Badge label={item.status} small />
        </View>
        <Text style={styles.title} numberOfLines={2}>
          {item.title}
        </Text>
        <Text style={styles.date}>{formatDate(item.createdAt)}</Text>
      </View>
    </Pressable>
  );
});


export default function NewsListScreen() {
  const router = useRouter();
  const { news, newsFilter, setNewsFilter } = useAdmin();
  const data = useMemo(() => filterNews(news, newsFilter), [news, newsFilter]);


  const filtersActive =
    newsFilter.status !== 'Усі' || newsFilter.category !== 'Усі' ||
newsFilter.query.trim() !== '';


  const openNews = useCallback((id) => router.push(`/news/${id}`), [router]);

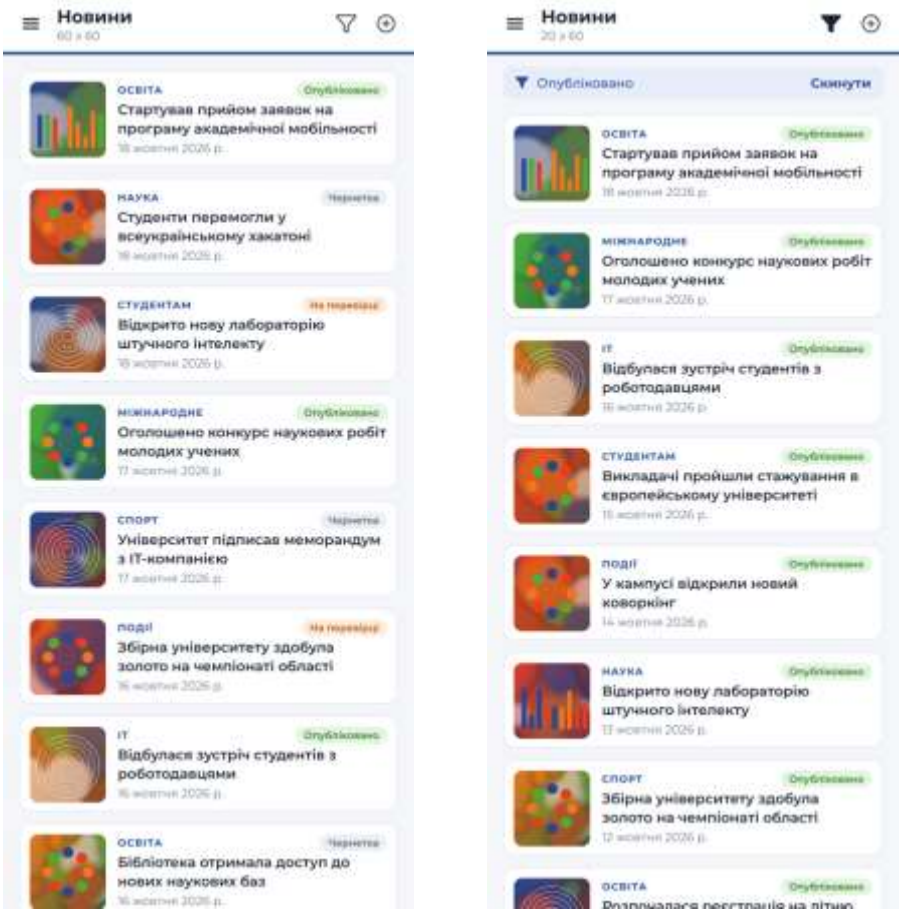

  return (
    <View style={{ flex: 1 }}>
      <Stack.Screen
        options={{
          headerSubtitle: `${data.length} з ${news.length}`,
          headerRight: () => (
            <>
              <HeaderIconButton icon={filtersActive ? 'funnel' : 'funnel-outline'}
label="Фільтр новин" onPress={() => router.push('/news-filter')} />
              <HeaderIconButton icon="add-circle-outline" label="Створити новину"
onPress={() => router.push('/news-form')} />
            </>
          ),
        }}
      />


      {filtersActive && (
        <View style={styles.filterBar}>
          <Ionicons name="funnel" size={14} color={colors.blue} />
          <Text style={styles.filterText} numberOfLines={1}>
```

```
            {[newsFilter.status, newsFilter.category, newsFilter.query &&
`«${newsFilter.query}»`]
              .filter((v) => v && v !== 'Усі')
              .join(' · ')}
          </Text>
          <Pressable onPress={() => setNewsFilter(DEFAULT_NEWS_FILTER)} hitSlop={8}>
            <Text style={styles.reset}>Скинути</Text>
          </Pressable>
        </View>
      )}

      <VirtualizedList
        data={data}
        getItemCount={(d) => d.length}
        getItem={(d, index) => d[index]}
        keyExtractor={(item) => item.id}
        getItemLayout={(_, index) => ({ length: ITEM_HEIGHT, offset: PADDING +
ITEM_HEIGHT * index, index })}
        renderItem={({ item }) => <NewsRow item={item} onPress={openNews} />}
        initialNumToRender={8}
        maxToRenderPerBatch={10}
        windowSize={9}
        contentContainerStyle={styles.list}
        ListEmptyComponent={<Text style={styles.empty}>Новин за заданими фільтрами не
знайдено</Text>}
      />
    </View>
  );
}

// ... оголошення стилів (StyleSheet.create) опущено
```



а) список новин б) відфільтрований список

Рисунок 3.3 - Розділ «Новини»

9. Екран деталей /news/[id]

Екран деталей отримує ідентифікатор новини хуком `useLocalSearchParams()` і показує зображення, статус, категорію, дату створення, текст і кнопки «Редагувати» та «Видалити». Якщо новину видалено (на цьому екрані або з модального вікна редагування), екран через `useFocusEffect` сам повертається до списку.

Лістинг 3.11 - app/(admin)/news/[id].jsx (без стилів)

```
import { Ionicons } from '@expo/vector-icons';
import { useFocusEffect, useLocalSearchParams, useRouter } from 'expo-router';
import { useCallback, useEffect, useRef } from 'react';
import { Image, ScrollView, StyleSheet, Text, useWindowDimensions, View } from 'react-
native';
import { Badge, confirmAction, PrimaryButton } from '../../../components/ui';
import { colors, fonts } from '../../../constants/theme';
import { useAdmin } from '../../../context/AdminContext';
import { formatDateTime } from '../../../data/seed';


export default function NewsDetailsScreen() {
  const { id } = useLocalSearchParams(); // параметр динамічного маршруту /news/[id]
  const router = useRouter();
```

```
  const { width } = useWindowDimensions();
  const { news, deleteNews } = useAdmin();
  const item = news.find((n) => n.id === id);


  // Якщо новину видалили (тут або з модального вікна редагування) -
  // повертаємося до списку, щойно цей екран у фокусі
  const existedRef = useRef(false);
  useEffect(() => {
    if (item) existedRef.current = true;
  }, [item]);
  useFocusEffect(
    useCallback(() => {
      if (!item && existedRef.current) {
        if (router.canGoBack()) router.back();
        else router.replace('/news');
      }
    }, [item, router])
  );


  if (!item) {
    return (
      <View style={styles.empty}>
        <Ionicons name="alert-circle-outline" size={40} color={colors.textMuted} />
        <Text style={styles.emptyText}>Новину з ID {id} не знайдено</Text>
      </View>
    );
  }


  const onDelete = () =>
    confirmAction('Видалити новину?', `«${item.title}» буде видалено без можливості
відновлення.`, () =>
      deleteNews(item.id)
    );


  return (
    <ScrollView contentContainerStyle={{ paddingBottom: 32 }}>
      <Image source={{ uri: item.imageUrl }} style={{ width, height: width * 0.56,
backgroundColor: '#DCE3EE' }} />
      <View style={styles.content}>
        <View style={styles.row}>
          <Badge label={item.status} />
          <Badge label={item.category} />
          <Text style={styles.id}>ID {item.id}</Text>
        </View>
        <Text style={styles.title}>{item.title}</Text>
        <View style={styles.row}>
          <Ionicons name="time-outline" size={14} color={colors.textMuted} />
          <Text style={styles.meta}>Створено: {formatDateTime(item.createdAt)}</Text>
        </View>
        <Text style={styles.text}>{item.text}</Text>
        <View style={styles.url}>
          <Ionicons name="image-outline" size={14} color={colors.textMuted} />
          <Text style={styles.urlText} numberOfLines={1}>
            {item.imageUrl}
          </Text>
        </View>


        <View style={styles.actions}>
          <PrimaryButton
            title="Редагувати"
            icon="create-outline"
            style={{ flex: 1 }}
            onPress={() => router.push({ pathname: '/news-form', params: { id: item.id
} })}
          />
          <PrimaryButton title="Видалити" icon="trash-outline" variant="danger"
style={{ flex: 1 }} onPress={onDelete} />
        </View>
      </View>
```

```
    </ScrollView>
  );
}

// ... оголошення стилів (StyleSheet.create) опущено
```

10. Створення та редагування новини (modal)

Форма зареєстрована з `presentation: 'modal'` і містить поля: заголовок, категорія та статус (вибір чипами), URL зображення з попереднім переглядом і текст новини. Перед збереженням виконується валідація; після збереження новина записується в масив (`createNews` або `updateNews`), а вікно закривається через `router.back()`. При відкритті з параметром `id` форма працює в режимі редагування.

Лістинг 3.12 - app/news-form.jsx (фрагмент)

```
export default function NewsFormModal() {
  const { id } = useLocalSearchParams();
  const router = useRouter();
  const insets = useSafeAreaInsets();
  const { news, createNews, updateNews, deleteNews } = useAdmin();
  // Знімок новини на момент відкриття: після видалення форма не "перемикається" на створення
  const [existing] = useState(() => (id ? news.find((n) => n.id === id) : null));

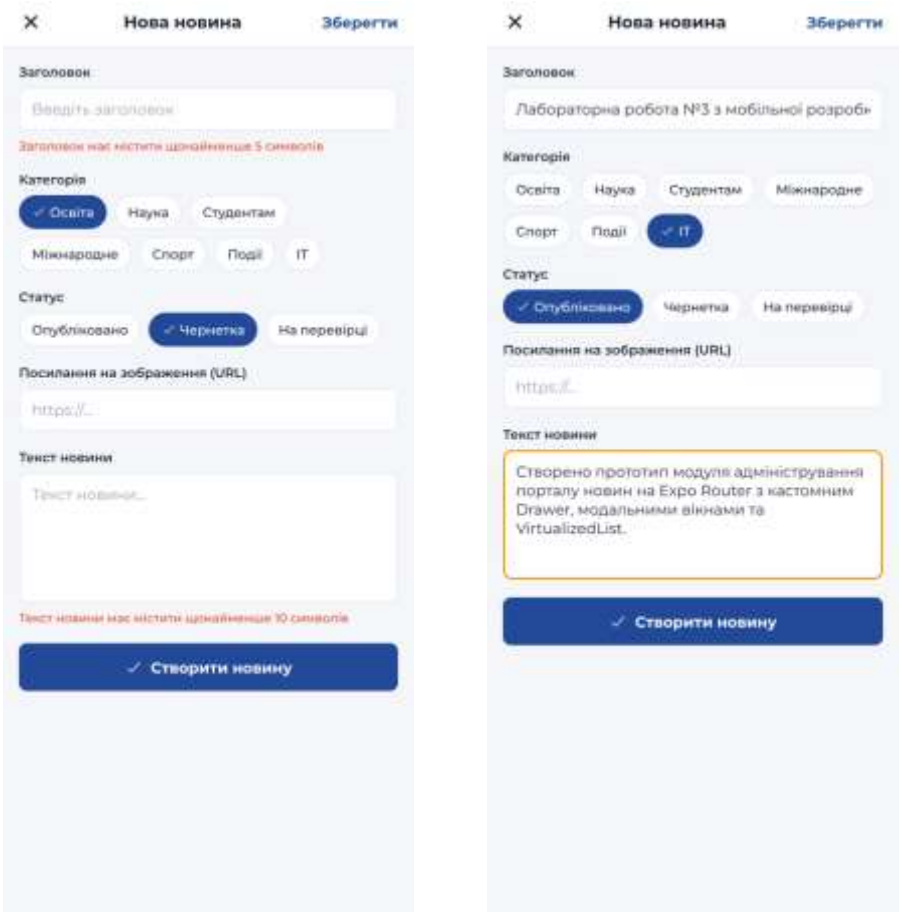
  const [form, setForm] = useState(() =>
    existing
      ? { title: existing.title, category: existing.category, status: existing.status, imageUrl: existing.imageUrl, text: existing.text }
      : EMPTY
  );
  const [errors, setErrors] = useState({});
  const set = (key) => (value) => {
    setForm((f) => ({ ...f, [key]: value }));
    setErrors((e) => ({ ...e, [key]: undefined }));
  };

  const validate = () => {
    const e = {};
    if (form.title.trim().length < 5) e.title = 'Заголовок має містити щонайменше 5 символів';
    if (form.imageUrl.trim() && !/^https?:\/\/\S+$/i.test(form.imageUrl.trim())) e.imageUrl = 'Посилання має починатися з http:// або https://';
    if (form.text.trim().length < 10) e.text = 'Текст новини має містити щонайменше 10 символів';
    setErrors(e);
    return Object.keys(e).length === 0;
  };

  const save = () => {
    if (!validate()) return;
    const data = {
      ...form,
      title: form.title.trim(),
      text: form.text.trim(),
```

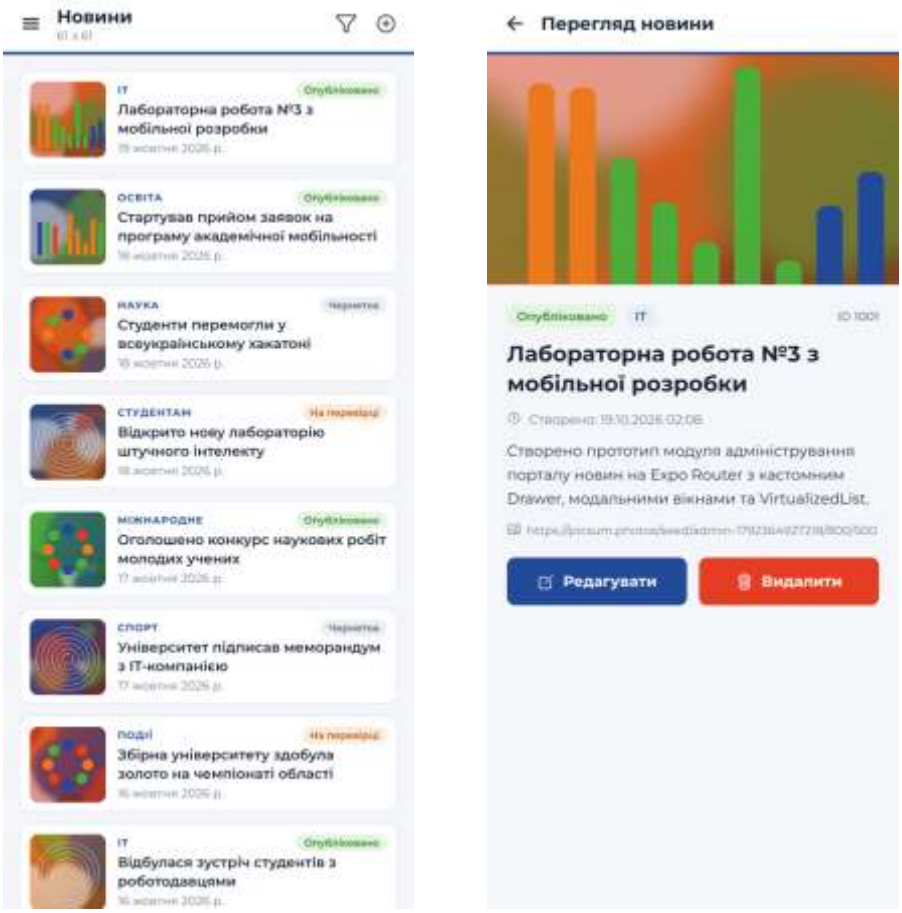
```
      imageUrl: form.imageUrl.trim() || `https://picsum.photos/seed/admin-
${Date.now()}/800/500`,
    };
    if (existing) updateNews(existing.id, data);
    else createNews(data);
    router.back(); // закриття модального вікна
  };


  const remove = () =>
    confirmAction('Видалити новину?', `«${existing.title}» буде видалено.`, () => {
      deleteNews(existing.id);
      router.back(); // екран деталей під модальним вікном сам повернеться до списку
    });
```



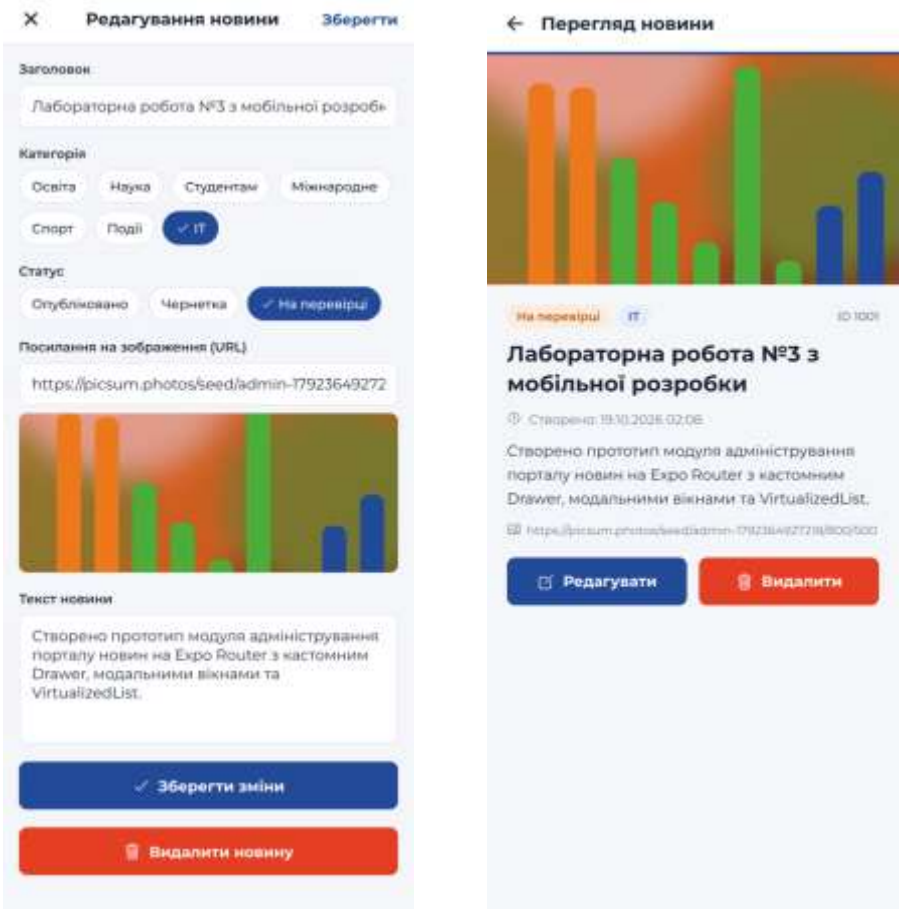
а) валідація полів б) заповнена форма

Рисунок 3.4 - Модальне вікно створення новини



а) новину додано б) деталі /news/1001

Рисунок 3.5 - Результат створення новини



а) редагування б) після збереження

Рисунок 3.6 - Редагування новини

11. Видалення з підтвердженням через Alert.alert()

Видалення новини підтверджується нативним діалогом `Alert.alert()` з кнопками «Скасувати» та «Видалити». Оскільки в web-версії (react-native-web) `Alert` не підтримується, для браузера використано `window.confirm`.

Лістинг 3.13 - components/ui.jsx (фрагмент: confirmAction)

```
// Нативний діалог підтвердження. Alert.alert не працює у web-версії
// react-native-web, тому для браузера використовується window.confirm.
export function confirmAction(title, message, onConfirm, confirmText = 'Видалити') {
  if (Platform.OS === 'web') {
    if (window.confirm(`${title}\n\n${message}`)) onConfirm();
    return;
  }
  Alert.alert(title, message, [
    { text: 'Скасувати', style: 'cancel' },
    { text: confirmText, style: 'destructive', onPress: onConfirm },
  ]);
}
```

12. Управління користувачами

Список користувачів також побудовано на `VirtualizedList` з параметрами оптимізації. Кожен рядок містить аватар з ініціалами, ім'я, пошту, роль і статус; кнопка праворуч блокує або розблоковує користувача (`toggleUserStatus`), що записується в журнал. Над списком показано кількість активних і заблокованих користувачів.

Лістинг 3.14 - app/(admin)/users.jsx (без стилів)

```
import { Ionicons } from '@expo/vector-icons';
import { useNavigation, useRouter } from 'expo-router';
import { memo, useLayoutEffect, useMemo } from 'react';
import { Pressable, StyleSheet, Text, View, VirtualizedList } from 'react-native';
import { Avatar, Badge, HeaderIconButton } from '../../components/ui';
import { colors, fonts, radius } from '../../constants/theme';
import { DEFAULT_USER_FILTER, filterUsers, useAdmin } from
'../../context/AdminContext';


const ITEM_HEIGHT = 92;
const CARD_HEIGHT = 82;
const PADDING = 16;
const ROLE_COLORS = { Адміністратор: colors.red, Редактор: colors.blue, Автор:
colors.orange, Читач: colors.green };


const UserRow = memo(function UserRow({ user, onToggle }) {
  const blocked = user.status === 'Заблокований';
  return (
    <View style={[styles.card, blocked && { opacity: 0.85 }]}>
      <Avatar name={user.name} size={44} color={ROLE_COLORS[user.role]} />
      <View style={{ flex: 1, gap: 3 }}>
        <Text style={styles.name} numberOfLines={1}>
          {user.name}
        </Text>
        <Text style={styles.email} numberOfLines={1}>
          {user.email}
        </Text>
        <View style={styles.badges}>
          <Text style={[styles.role, { color: ROLE_COLORS[user.role]
}]}>{user.role}</Text>
          <Badge label={user.status} small />
        </View>
      </View>
      <Pressable
        onPress={() => onToggle(user.id)}
        accessibilityLabel={blocked ? `Розблокувати ${user.name}` : `Заблокувати
${user.name}`}
        style={({ pressed }) => [styles.toggle, blocked ? styles.toggleUnblock :
styles.toggleBlock, pressed && { opacity: 0.7 }]}
      >
        <Ionicons name={blocked ? 'lock-open-outline' : 'ban-outline'} size={16}
color={blocked ? colors.green : colors.red} />
        <Text style={[styles.toggleText, { color: blocked ? colors.green : colors.red
}]}>
          {blocked ? 'Розблок.' : 'Блок.'}
        </Text>
      </Pressable>
    </View>
  );
});
```

```
export default function UsersScreen() {
  const router = useRouter();
  const navigation = useNavigation();
  const { users, userFilter, setUserFilter, toggleUserStatus } = useAdmin();
  const data = useMemo(() => filterUsers(users, userFilter), [users, userFilter]);
  const filtersActive = userFilter.role !== 'Усі' || userFilter.status !== 'Усі' ||
userFilter.query.trim() !== '';

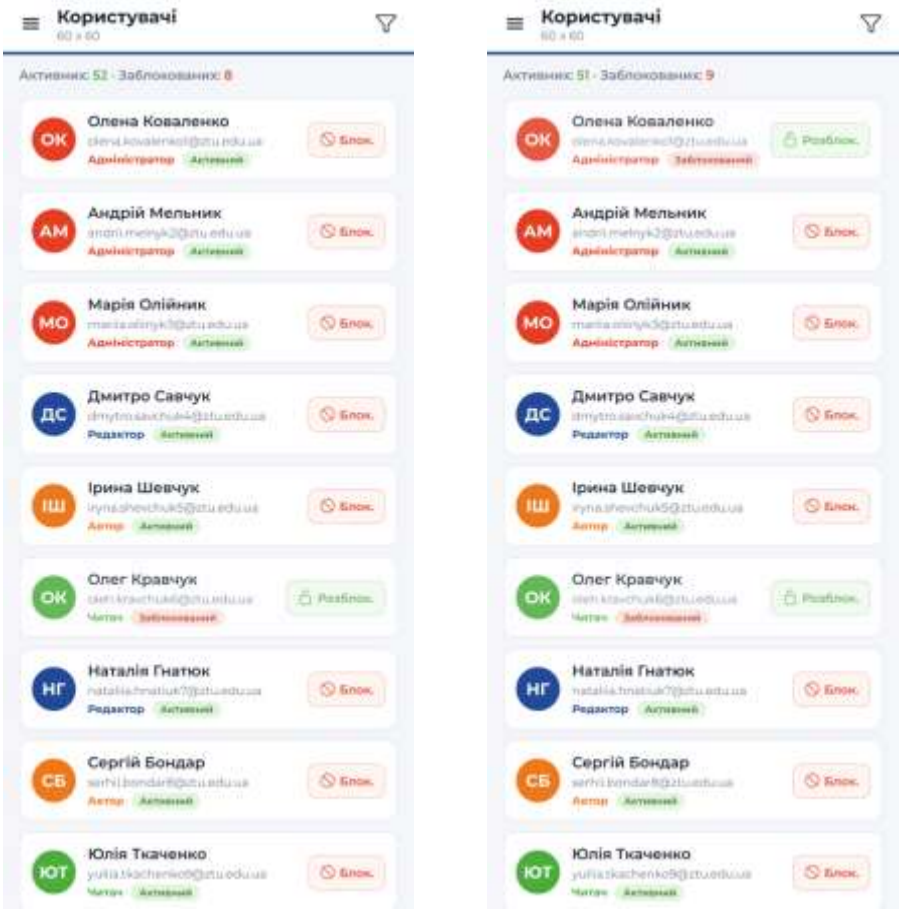

  useLayoutEffect(() => {
    navigation.setOptions({
      headerSubtitle: `${data.length} з ${users.length}`,
      headerRight: () => (
        <HeaderIconButton icon={filtersActive ? 'funnel' : 'funnel-outline'}
label="Фільтр користувачів" onPress={() => router.push('/users-filter')} />
      ),
    });
  }, [navigation, router, data.length, users.length, filtersActive]);


  const active = users.filter((u) => u.status === 'Активний').length;


  return (
    <View style={{ flex: 1 }}>
      <View style={styles.summary}>
        <Text style={styles.summaryText}>
          Активних: <Text style={{ color: colors.green }}>{active}</Text> ·
Заблокованих:{' '}
          <Text style={{ color: colors.red }}>{users.length - active}</Text>
        </Text>
        {filtersActive && (
          <Pressable onPress={() => setUserFilter(DEFAULT_USER_FILTER)}>
            <Text style={styles.reset}>Скинути фільтр</Text>
          </Pressable>
        )}
      </View>


      <VirtualizedList
        data={data}
        getItemCount={(d) => d.length}
        getItem={(d, index) => d[index]}
        keyExtractor={(item) => item.id}
        getItemLayout={(_, index) => ({ length: ITEM_HEIGHT, offset: PADDING +
ITEM_HEIGHT * index, index })}
        renderItem={({ item }) => <UserRow user={item} onToggle={toggleUserStatus} />}
        initialNumToRender={10}
        maxToRenderPerBatch={10}
        windowSize={9}
        contentContainerStyle={styles.list}
        ListEmptyComponent={<Text style={styles.empty}>Користувачів не
знайдено</Text>}
      />
    </View>
  );
}


// ... оголошення стилів (StyleSheet.create) опущено
```



а) список користувачів б) зміна статусу

Рисунок 3.7 - Розділ «Користувачі»

13. Журнал системних логів

Журнал виводиться на `VirtualizedList` від найновіших записів до найстаріших; над списком є чипи для фільтрації за типом події. Кожен запис містить іконку типу, назву події, деталі, автора та час.

Лістинг 3.15 - app/(admin)/logs.jsx (фрагмент)

```
export default function LogsScreen() {
  const navigation = useNavigation();
  const { logs } = useAdmin();
  const [type, setType] = useState('ALL');
  const data = useMemo(() => (type === 'ALL' ? logs : logs.filter((l) => l.type ===
type)), [logs, type]);

  useLayoutEffect(() => {
    navigation.setOptions({ headerSubtitle: `${logs.length} записів` });
  }, [navigation, logs.length]);

  return (
    <View style={{ flex: 1 }}>
      <ScrollView horizontal showsHorizontalScrollIndicator={false}
contentContainerStyle={styles.chips} style={{ flexGrow: 0 }}>
        {[['ALL', 'Усі'], ...Object.entries(LOG_TYPES).map(([k, v]) => [k,
v.label])].map(([key, label]) => (
```

```
          <Pressable key={key} onPress={() => setType(key)} style={[styles.chip, type
=== key && styles.chipActive]}>
            <Text style={[styles.chipText, type === key && { color: '#fff'
}]}>{label}</Text>
          </Pressable>
        ))}
      </ScrollView>


      <VirtualizedList
        data={data}
        getItemCount={(d) => d.length}
        getItem={(d, index) => d[index]}
        keyExtractor={(item) => item.id}
        getItemLayout={(_, index) => ({ length: ITEM_HEIGHT, offset: PADDING +
ITEM_HEIGHT * index, index })}
        renderItem={({ item }) => <LogRow log={item} />}
        initialNumToRender={12}
        maxToRenderPerBatch={12}
        windowSize={9}
        contentContainerStyle={styles.list}
        ListEmptyComponent={<Text style={styles.empty}>Записів цього типу
немає</Text>}
      />
    </View>
  );
}
```

14. Панелі фільтрації (formSheet)

Фільтри новин (статус, категорія, пошук за заголовком) і користувачів (роль, статус, пошук за ім’ям або поштою) відкриваються як нижні панелі з `presentation: 'formSheet'`. Висоту панелі задано через `sheetAllowedDetents: [0.6, 0.95]`, тому користувач може перетягувати її між двома положеннями. Зміни застосовуються до чернетки й переносяться в глобальний стан лише після натискання «Застосувати».

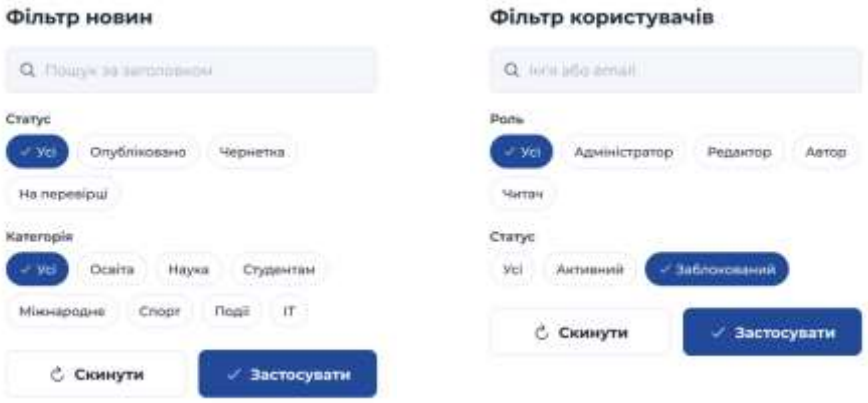
Лістинг 3.16 - app/news-filter.jsx

```
import { useRouter } from 'expo-router';
import { useState } from 'react';
import { FilterSheet } from '../components/FilterSheet';
import { DEFAULT_NEWS_FILTER, useAdmin } from '../context/AdminContext';
import { NEWS_CATEGORIES, NEWS_STATUSES } from '../data/seed';


// Панель фільтрації новин (presentation: 'formSheet', sheetAllowedDetents)
export default function NewsFilterSheet() {
  const router = useRouter();
  const { newsFilter, setNewsFilter } = useAdmin();
  const [draft, setDraft] = useState(newsFilter);
  const set = (key) => (value) => setDraft((d) => ({ ...d, [key]: value }));

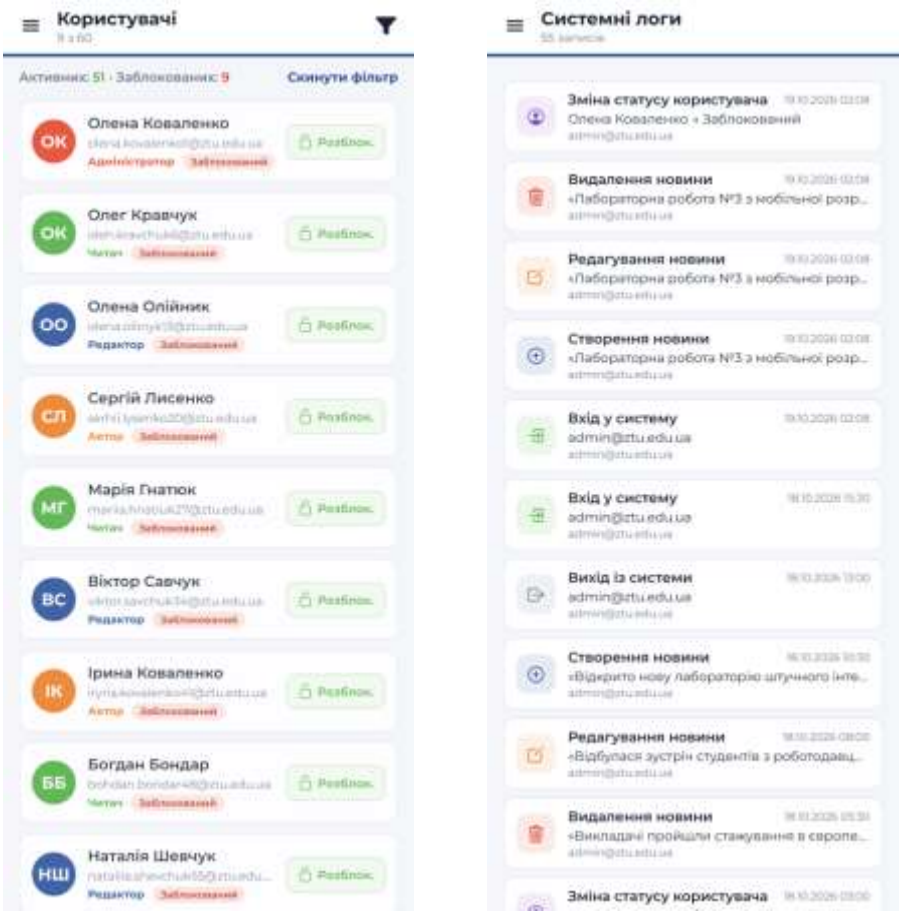

  return (
    <FilterSheet
      title="Фільтр новин"
      query={draft.query}
      onQuery={set('query')}
      queryPlaceholder="Пошук за заголовком"
      groups={[
```

```
        { label: 'Статус', options: ['Усі', ...NEWS_STATUSES], value: draft.status,
onChange: set('status') },
        { label: 'Категорія', options: ['Усі', ...NEWS_CATEGORIES], value:
draft.category, onChange: set('category') },
      ]}
      onReset={() => setDraft(DEFAULT_NEWS_FILTER)}
      onApply={() => {
        setNewsFilter(draft);
        router.back();
      }}
    />
  );
}
```



а) фільтр новин б) фільтр користувачів

Рисунок 3.8 - Панелі фільтрації

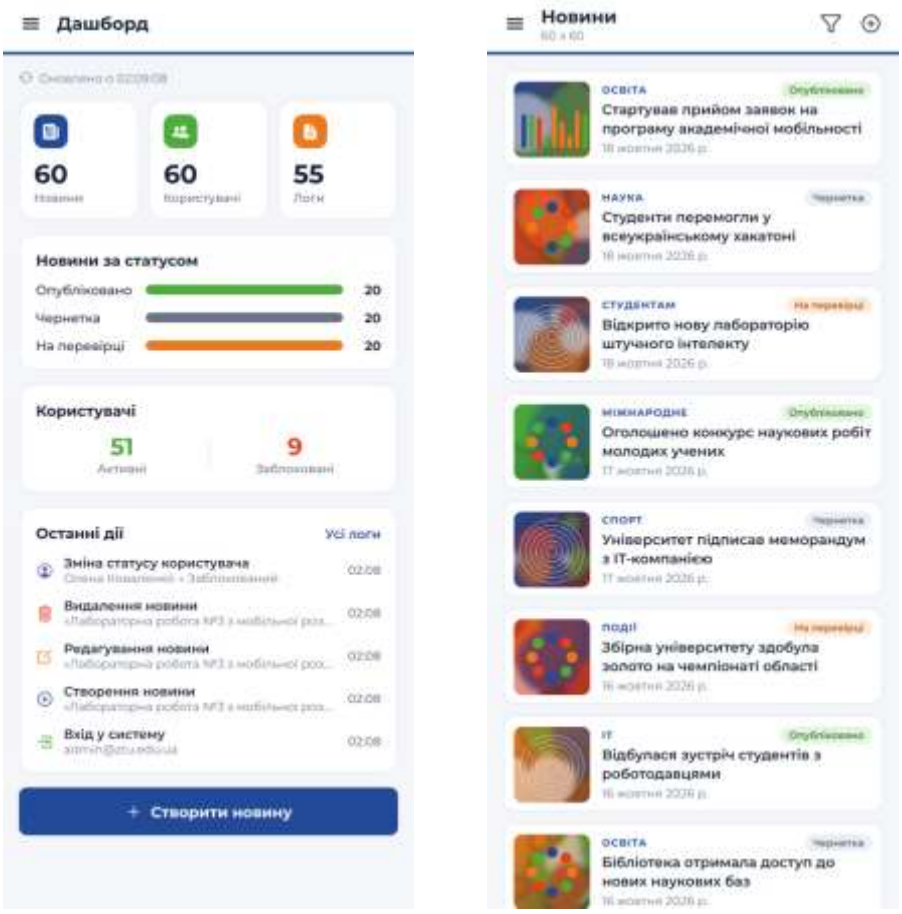


а) заблоковані користувачі б) журнал після дій

Рисунок 3.9 - Результат фільтрації та журнал логів

15. Запуск, тестування та розміщення на GitHub

Застосунок запускався командою `npx expo start` на смартфоні через Expo Go та в емуляторі Android. Перевірено захист маршрутів, вхід і вихід, роботу фільтрів, створення новини з валідацією, редагування, видалення з підтвердженням і зі скасуванням, блокування користувачів, оновлення дашборда при поверненні на екран, а також те, що журнал поповнюється рівно на шість визначених подій.



а) дашборд після дій б) список після видалення

Рисунок 3.10 - Актуалізація даних після операцій

Вихідний код розміщено у репозиторії https://github.com/ipz242dmv-alt/MobileLabsRN2026 у папці lab3 разом із README.md.

ВІДПОВІДІ НА КОНТРОЛЬНІ ЗАПИТАННЯ

1. Яким чином виконується кастомізація бічного меню та хедера стекової навігації за допомогою drawerContent та screenOptions.header?

Проп `drawerContent` навігатора `<Drawer>` приймає функцію, яка отримує `props` (`state`, `navigation`, `descriptors`) і повертає власний компонент замість стандартного списку пунктів. У `AdminDrawerContent` над пунктами розміщено блок профілю, пункти виводяться стандартним `DrawerItemList` усередині `DrawerContentScrollView` (тому працюють підсвічування активного пункту та іконки з опцій екранів), а кнопка «Вийти» закріплена внизу. Опція `screenOptions.header` повністю замінює

стандартний хедер: функція отримує `{ navigation, route, options, back }` і повертає довільну розмітку. Той самий компонент `AdminHeader` передано і в Drawer, і у вкладений Stack, а окремі екрани доповнюють його через `navigation.setOptions()` або `<Stack.Screen options={...} />`.

2. У чому відмінність між нативними режимами модальних вікон presentation: 'modal' та presentation: 'formSheet'?

`modal` - повноцінне модальне вікно для окремого завдання: на iOS це картка, що виїжджає знизу й майже повністю закриває попередній екран, на Android - новий екран поверх поточного. Підходить для форм, які потребують повної уваги, як створення чи редагування новини. `formSheet` - нативна нижня панель, висота якої задається `sheetAllowedDetents` (частки висоти екрана або `'fitToContents'`); користувач може перетягувати її між положеннями, а попередній екран частково лишається видимим. Додатково налаштовуються `sheetGrabberVisible`, `sheetInitialDetentIndex`, `sheetCornerRadius`. Цей режим зручний для допоміжних дій, що не відривають від контексту, наприклад фільтрів.

3. Для чого використовується хук useFocusEffect і чим він відрізняється від стандартного useEffect?

У навігаторах екрани зазвичай не демонтуються при переході: дашборд лишається змонтованим, поки користувач працює в інших розділах або модальних вікнах. Тому `useEffect(..., [])` виконається лише при першому монтуванні. `useFocusEffect` запускає колбек щоразу, коли екран отримує фокус, а функцію очищення - при втраті фокусу. Колбек обгортають у `useCallback`, щоб ефект не перезапускався на кожному рендері. На дашборді в ньому перераховується статистика; типові сценарії - оновлення даних, підписки, таймери, які потрібні лише видимому екрану.

ВИСНОВКИ

У ході лабораторної роботи створено прототип модуля адміністрування порталу новин на файловій маршрутизації Expo Router. Побудовано ієрархію

навігації: кореневий Stack із захищеними групами (`Stack.Protected`), Drawer для службових розділів і вкладений Stack для новин із динамічним маршрутом `/news/[id]`. Бічне меню та хедер кастомізовано через `drawerContent` і `screenOptions.header`. Для створення, редагування та фільтрації використано нативні режими `modal` і `formSheet` з `sheetAllowedDetents`, а видалення підтверджується нативним діалогом `Alert.alert()`. Списки новин, користувачів і логів реалізовано на `VirtualizedList` з `getItemCount`, `getItem` і `getItemLayout`, що дає змогу не вимірювати елементи та миттєво обчислювати їхнє положення. Журнал фіксує лише визначені системні події, а дашборд актуалізує дані через `useFocusEffect`.